

Практика 3. Типы в λ -исчислении

Андрей Стоян, Илья Колегов, Дмитрий Халанский

MSE ITMO

осень 2025

В предыдущих сериях

- ▶ Синтаксис и семантика λ -исчисления
- ▶ Программирование на λ -исчислении!
- + Синтаксис типов и термов $\lambda_{\rightarrow}$
- + Правила типизации $\lambda_{\rightarrow}$
- + Гарантии системы $\lambda_{\rightarrow}$

Содержание

1. Идея систем типов
2. $\lambda_{\rightarrow}$ интуитивно
3. $\lambda_{\rightarrow}$ формально
 - ▶ Задача проверки типа
 - ▶ Задача вывода типа
 - ▶ Задача населения типа
 - ▶ Стили типизации
4. Материалы (☁)

Содержание

1. Идея систем типов

2. $\lambda \rightarrow$ интуитивно

3. $\lambda \rightarrow$ формально

- ▶ Задача проверки типа
- ▶ Задача вывода типа
- ▶ Задача населения типа
- ▶ Стили типизации

4. Материалы (☀)

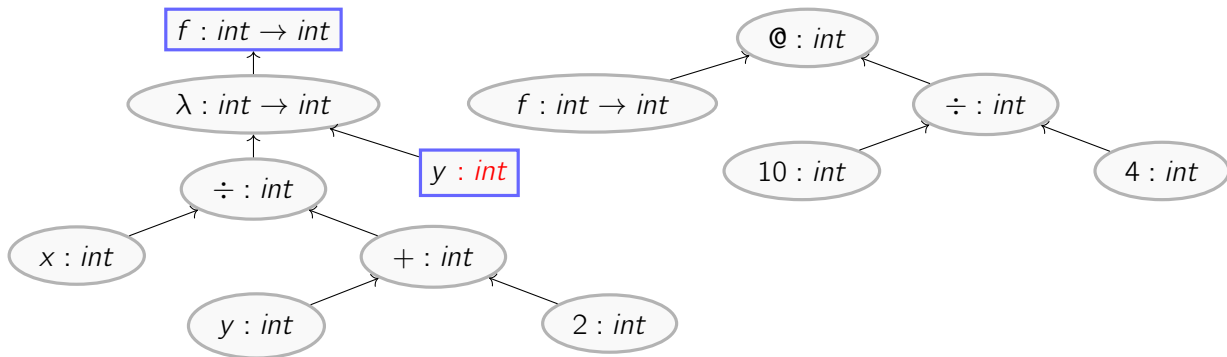
Мотивация статического анализа программ

- ▶ Программное обеспечение — это сложно
- ▶ В программах случаются ошибки (постоянно и неизбежно)
- ▶ Поиск ошибок может занимать годы, десятилетия, вечность
- ▶ Нужны методы для того, чтобы судить о программах и их свойствах автоматически
- ▶ Программы могут работать долго
- ▶ Хотим отвергать “плохие” программы до исполнения (статически)

Анализ типов

Идея анализа типов

- ▶ Попытаемся приписать каждой ноде программы синтаксические метки по определённым правилам
- ▶ Если метки таким образом приписать можно — программа “хорошая”
- ▶ Вид таких меток и правил задаётся *системой типов*
- ▶ Анализ типов — разновидность статического анализа



Роль типов в языках

- ▶ Позволяют выявлять некоторые классы ошибок:

```
GHCi> length 42          -- Ошибка: на вход не список, а число
```

```
GHCi> foldr (+) "ABCDE" -- Ошибка: плюс работает на числах, но не символах
```

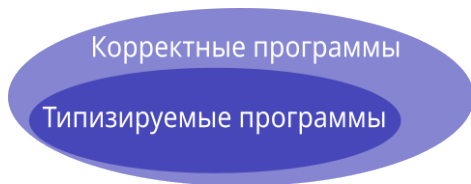
- ▶ Механизм абстракции: интерфейс, скрывающий детали реализации
- ▶ Средство документирования: как использовать сущность
- ▶ Частичная спецификация поведения
- ▶ Позволяют генерировать более эффективный низкоуровневый код

Ограниченность систем типов (☁)

- ▶ Хотелось бы, чтобы все “хорошие” программы типизировались, а все “плохие” — нет
- ▶ Однако по теореме Успенского-Райса задача проверки любого нетривиального¹ свойства программы неразрешима

В чем подвох с системами типов?

- ▶ Всё, что типизируется, — “хорошее”, но обратное не всегда верно
- ⇒ Всегда существуют “хорошие” программы, которые не типизируются²

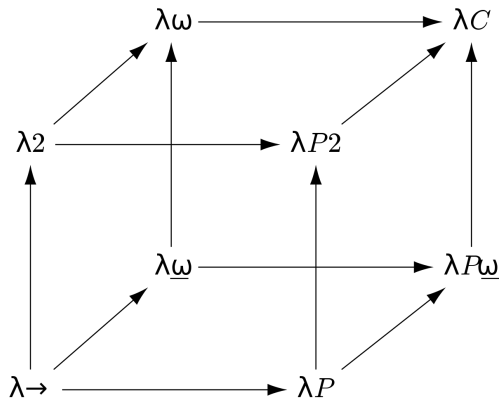


¹Существуют программы, как обладающие таким свойством, так и не обладающие.

²На практике к тому же зачастую существуют “плохие”, которые типизируются.

При чём тут ФП? ☀️

- ▶ Парадигме агрессивного абстрагирования требуются гибкие системы типов
- ▶ ФП языки минималистичные и регулярные — с ними проще работать



Содержание

1. Идея систем типов
2. $\lambda \rightarrow$ интуитивно
3. $\lambda \rightarrow$ формально
 - ▶ Задача проверки типа
 - ▶ Задача вывода типа
 - ▶ Задача населения типа
 - ▶ Стили типизации
4. Материалы (☀)

$\lambda \rightarrow$ типизация интуитивно (1)

todo Припишите тип терму $\lambda x^\alpha. x$

- ▶ Терм задаёт функцию — тип стрелочный: $\alpha \rightarrow ?$
- ▶ Типизируем тело, зная $\{x : \alpha\}$: $\lambda x^\alpha. \underbrace{x}$
- ▶ Тип результата функции — α (уже знаем тип x)
- ▶ Значит, тип всей функции $\alpha \rightarrow \alpha$

todo Протипизируйте $f x$, зная, что $\{x : \alpha, f : \alpha \rightarrow \beta\}$

- ▶ Тип x совпадает с ожидаемым типом аргумента
- ▶ Тип применения $f x$ — тип результата функции β

todo Припишите тип терму $\lambda x^\alpha f^{\alpha \rightarrow \beta}. f x$

- ▶ Видим абстракцию — пишем стрелку $\alpha \rightarrow ?$
- ▶ Типизируем тело, зная $\{x : \alpha\}$: $\lambda x^\alpha. \underbrace{\lambda f^{\alpha \rightarrow \beta}. f x}$
- ▶ Тип тела — тоже функция: $\alpha \rightarrow ((\alpha \rightarrow \beta) \rightarrow ?)$
- ▶ Типизируем тело вложенной функции, зная $\{x : \alpha, f : \alpha \rightarrow \beta\}$: $\lambda x^\alpha. \underbrace{\lambda f^{\alpha \rightarrow \beta}. f x}$
- ▶ Уже решали, итоговый тип: $\alpha \rightarrow ((\alpha \rightarrow \beta) \rightarrow \beta)$
- ▶ По ассоциативности стрелки можно убрать одни скобки: $\alpha \rightarrow (\alpha \rightarrow \beta) \rightarrow \beta$

$\lambda \rightarrow$ типизация интуитивно (2)

todo Припишите тип терму $\lambda x y. x$



- ▶ Типы аргументов не записаны в терме — выбираем их самостоятельно
- ▶ Тело терма никак типы аргументов не ограничивает (нет аппликаций)
- ▶ Выбираем различные буквы: $\alpha \rightarrow \beta \rightarrow ?$
- ▶ Тип результата совпадает с типом первого аргумента
- ▶ Ответ: $\alpha \rightarrow \beta \rightarrow \alpha$

todo Припишите тип терму $\lambda x y f. f x y$



- ▶ f в теле принимает два аргумента
- ▶ Типы x и y никак не ограничены
- ▶ $\alpha \rightarrow \beta \rightarrow (\alpha \rightarrow \beta \rightarrow ?) \rightarrow ?$
- ▶ Тип результата функции ничем не ограничен — берём свежую букву
- ▶ Ответ: $\alpha \rightarrow \beta \rightarrow (\alpha \rightarrow \beta \rightarrow \gamma) \rightarrow \gamma$

$\lambda \rightarrow$ население типов интуитивно

todo Населите тип $(\beta \rightarrow \gamma) \rightarrow (\alpha \rightarrow \beta) \rightarrow \alpha \rightarrow \gamma$



- ▶ Вспоминаем про ассоциативность стрелки: $(\beta \rightarrow \gamma) \rightarrow ((\alpha \rightarrow \beta) \rightarrow \alpha \rightarrow \gamma)$
- ▶ Это явно функция, пишем абстрактор: $\lambda f^{\beta \rightarrow \gamma}.$

- ▶ Аналогично ещё две стрелки дают два аргумента: $\lambda f^{\beta \rightarrow \gamma} \overbrace{g^{\alpha \rightarrow \beta}}^{(\alpha \rightarrow \beta) \rightarrow \alpha \rightarrow \gamma} \underbrace{x^\alpha}_{\gamma}.$

- ▶ Единственный способ получить значение типа γ — вызвать f :
 $\lambda f^{\beta \rightarrow \gamma} \overbrace{g^{\alpha \rightarrow \beta}}^{\beta} \underbrace{x^\alpha}_{\gamma}. f$

- ▶ Конструируем подходящий аргумент для f типа β вызовом g :
 $\lambda f^{\beta \rightarrow \gamma} \overbrace{g^{\alpha \rightarrow \beta}}^{\beta} \underbrace{x^\alpha}_{\gamma}. f (g \underbrace{\quad}_{\alpha})$

- ▶ Воспользуемся x , чтобы достроить терм нужного типа:
 $\lambda f^{\beta \rightarrow \gamma} \overbrace{g^{\alpha \rightarrow \beta}}^{\beta} \underbrace{x^\alpha}_{\gamma}. f (g x)$

Содержание

1. Идея систем типов

2. $\lambda \rightarrow$ интуитивно

3. $\lambda \rightarrow$ формально

- ▶ Задача проверки типа
- ▶ Задача вывода типа
- ▶ Задача населения типа
- ▶ Стили типизации

4. Материалы (☀)

Контекст типизации

- ▶ В программах используются введённые ранее имена
- ▶ Для типизации нужно знать типы сущностей, связанных ранее с этими именами
- ▶ Будем поддерживать доп. структуру данных, которую будем заполнять именами с соответствующими им типами в процессе обхода дерева программы

Пример на Си

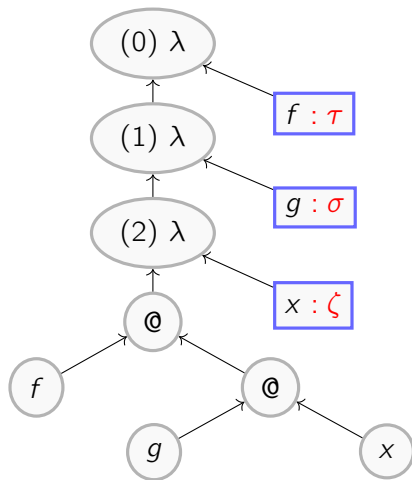
```
int dostrange(int a, char i) {  
    // point 0  
    int c = 5;  
    for (int i = 0; i < 15; ++i) {  
        i -= 1; // point 1  
        long r = i;  
        i *= r; // point 2  
    }  
    return a + (int)i * c; // point 3  
}
```

Контекст типизации в каждой из точек:

0. $\{a : \text{int}, i : \text{char}\}$
 1. $\{a : \text{int}, c : \text{int}, i : \text{int}\}$
 2. $\{a : \text{int}, c : \text{int}, i : \text{int}, r : \text{long}\}$
 3. $\{a : \text{int}, c : \text{int}, i : \text{char}\}$
- + Глобальные определения: $<$, $+$, $*$...

Контекст типизации: пример на лямбда-исчислении

В чистом лямбда-исчислении имена вводятся с помощью абстракции.



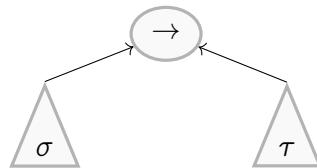
Контекст типизации для каждой вершины:

- 0. $\emptyset$
- 1. $\{f : \tau\}$
- 2. $\{f : \tau, g : \sigma\}$
- *. $\{f : \tau, g : \sigma, x : \zeta\}$

Определение STLC / simply typed λ -calculus / $\lambda_{\rightarrow}$

Синтаксис типов в $\lambda_{\rightarrow}$

Типовые переменные $\alpha, \beta, \dots \in \mathbb{T}$



Стрелочные типы $\sigma, \tau \in \mathbb{T} \Rightarrow (\sigma \rightarrow \tau) \in \mathbb{T}^1$

¹Стрелочка — правоассоциативный оператор: $\tau \rightarrow (\sigma \rightarrow \zeta) = \tau \rightarrow \sigma \rightarrow \zeta$.

Утверждение типизации

- ▶ Синтаксис: Γ — контекст, $M \in \Lambda$, $\sigma \in \mathbb{T}$, то $\Gamma \vdash M : \sigma$ — утверждение типизации
- ▶ Синтаксическая категория, увязывающая вместе термы и типы с контекстом
- ▶ Утверждает, что в контексте Γ терм M имеет тип σ

Правила вывода утверждений типизации в $\lambda_{\rightarrow}$

$$\frac{(x : \sigma) \in \Gamma}{\Gamma \vdash x : \sigma} \text{ ctx} \quad \frac{\Gamma \vdash M : \sigma \rightarrow \tau \quad \Gamma \vdash N : \sigma}{\Gamma \vdash M N : \tau} \text{ elim} \rightarrow \quad \frac{\{x : \sigma\} \cup \Gamma \vdash M : \tau}{\Gamma \vdash \lambda x^{\sigma}. M : \sigma \rightarrow \tau} \text{ intro} \rightarrow$$

Содержание

1. Идея систем типов
2. $\lambda \rightarrow$ интуитивно
3. $\lambda \rightarrow$ формально
 - ▶ Задача проверки типа
 - ▶ Задача вывода типа
 - ▶ Задача населения типа
 - ▶ Стили типизации
4. Материалы (☁)

Проверка типа комбинатора **B** в $\lambda \rightarrow$

Правила вывода утверждений типизации в $\lambda \rightarrow$

$$\frac{(x : \sigma) \in \Gamma}{\Gamma \vdash x : \sigma} \text{ ctx} \quad \frac{\Gamma \vdash M : \sigma \rightarrow \tau \quad \Gamma \vdash N : \sigma}{\Gamma \vdash M N : \tau} \text{ elim} \rightarrow \quad \frac{\{x : \sigma\} \cup \Gamma \vdash M : \tau}{\Gamma \vdash \lambda x^{\sigma}. M : \sigma \rightarrow \tau} \text{ intro} \rightarrow$$

$$\begin{array}{c} \frac{\overline{\Gamma \vdash f : \beta \rightarrow \gamma} \text{ ctx} \quad \frac{\overline{\Gamma \vdash g : \alpha \rightarrow \beta} \text{ ctx} \quad \overline{\Gamma \vdash x : \alpha} \text{ ctx}}{\Gamma \vdash g x : \beta} \text{ elim} \rightarrow}{\{f : \beta \rightarrow \gamma, g : \alpha \rightarrow \beta, x : \alpha\} \vdash f (g x) : \gamma} \text{ elim} \rightarrow \\ \frac{\{f : \beta \rightarrow \gamma, g : \alpha \rightarrow \beta\} \vdash \lambda x. f (g x) : \alpha \rightarrow \gamma}{\{f : \beta \rightarrow \gamma\} \vdash \lambda g x. f (g x) : (\alpha \rightarrow \beta) \rightarrow \alpha \rightarrow \gamma} \text{ intro} \rightarrow \\ \frac{\{f : \beta \rightarrow \gamma\} \vdash \lambda g x. f (g x) : (\alpha \rightarrow \beta) \rightarrow \alpha \rightarrow \gamma}{\emptyset \vdash \lambda f g x. f (g x) : (\beta \rightarrow \gamma) \rightarrow (\alpha \rightarrow \beta) \rightarrow \alpha \rightarrow \gamma} \text{ intro} \rightarrow \end{array}$$

- Получилось перевернутое синтаксическое дерево терма!

Таинственные связи (☁)

Дерево вывода типа комбинатора **B** или вывод силлогизма в исчислении высказываний?

$$\begin{array}{c}
 \frac{\overline{\Gamma \vdash f : \beta \rightarrow \gamma} \text{ InCtx} \quad \frac{\overline{\Gamma \vdash g : \alpha \rightarrow \beta} \text{ InCtx} \quad \overline{\Gamma \vdash x : \alpha} \text{ InCtx}}{\Gamma \vdash g x : \beta} \text{ MP}}{\{f : \beta \rightarrow \gamma, g : \alpha \rightarrow \beta, x : \alpha\} \vdash f (g x) : \gamma} \text{ MP} \\
 \frac{\{f : \beta \rightarrow \gamma, g : \alpha \rightarrow \beta\} \vdash \lambda x. f (g x) : \alpha \rightarrow \gamma}{\{f : \beta \rightarrow \gamma\} \vdash \lambda g x. f (g x) : (\alpha \rightarrow \beta) \rightarrow \alpha \rightarrow \gamma} \text{ intro } \rightarrow \\
 \frac{\{f : \beta \rightarrow \gamma\} \vdash \lambda g x. f (g x) : (\alpha \rightarrow \beta) \rightarrow \alpha \rightarrow \gamma}{\emptyset \vdash \lambda f g x. f (g x) : (\beta \rightarrow \gamma) \rightarrow (\alpha \rightarrow \beta) \rightarrow \alpha \rightarrow \gamma} \text{ intro } \rightarrow
 \end{array}$$

Содержание

1. Идея систем типов
2. $\lambda \rightarrow$ интуитивно
3. $\lambda \rightarrow$ формально
 - ▶ Задача проверки типа
 - ▶ **Задача вывода типа**
 - ▶ Задача населения типа
 - ▶ Стили типизации
4. Материалы (☀)

Вывод типа для комбинатора **B** (0)

Правила вывода утверждений типизации в $\lambda \rightarrow$

$$\frac{(x : \sigma) \in \Gamma}{\Gamma \vdash x : \sigma} \text{ ctx} \quad \frac{\Gamma \vdash M : \sigma \rightarrow \tau \quad \Gamma \vdash N : \sigma}{\Gamma \vdash M N : \tau} \text{ elim} \rightarrow \quad \frac{\{x : \sigma\} \cup \Gamma \vdash M : \tau}{\Gamma \vdash \lambda x^{\sigma}. M : \sigma \rightarrow \tau} \text{ intro} \rightarrow$$

$$\emptyset \vdash \lambda f g x. f (g x) : ?_0$$

- ▶ Требуется вывести тип терма
- ▶ Внешней информации нет, типизируем в пустом контексте
- ▶ Пока для неизвестного типа используем мета-переменную $?_0$

Вывод типа для комбинатора **B** (1)

Правила вывода утверждений типизации в $\lambda \rightarrow$

$$\frac{(x : \sigma) \in \Gamma}{\Gamma \vdash x : \sigma} \text{ ctx} \quad \frac{\Gamma \vdash M : \sigma \rightarrow \tau \quad \Gamma \vdash N : \sigma}{\Gamma \vdash M N : \tau} \text{ elim} \rightarrow \quad \frac{\{x : \sigma\} \cup \Gamma \vdash M : \tau}{\Gamma \vdash \lambda x^{\sigma}. M : \sigma \rightarrow \tau} \text{ intro} \rightarrow$$

$$\frac{\{f : ?_1\} \vdash \lambda g x. f (g x) : ?_2}{\emptyset \vdash \lambda f g x. f (g x) : ?_1 \rightarrow ?_2} \text{ intro} \rightarrow$$

- ▶ Видим абстракцию на верхнем уровне: правило $\text{intro} \rightarrow$
- ▶ Правило требует стрелочной структуры типа, запоминаем $?_0 = ?_1 \rightarrow ?_2$
- ▶ Обогащаем контекст переменной типа аргумента функции

Вывод типа для комбинатора **B** (2)

Правила вывода утверждений типизации в $\lambda \rightarrow$

$$\frac{(x : \sigma) \in \Gamma}{\Gamma \vdash x : \sigma} \text{ ctx} \quad \frac{\Gamma \vdash M : \sigma \rightarrow \tau \quad \Gamma \vdash N : \sigma}{\Gamma \vdash M N : \tau} \text{ elim} \rightarrow \quad \frac{\{x : \sigma\} \cup \Gamma \vdash M : \tau}{\Gamma \vdash \lambda x^{\sigma}. M : \sigma \rightarrow \tau} \text{ intro} \rightarrow$$

$$\frac{\frac{\{f : ?_1, g : ?_3\} \vdash \lambda x. f (g x) : ?_4}{\{f : ?_1\} \vdash \lambda g x. f (g x) : ?_3 \rightarrow ?_4} \text{ intro} \rightarrow}{\emptyset \vdash \lambda f g x. f (g x) : ?_1 \rightarrow ?_3 \rightarrow ?_4} \text{ intro} \rightarrow$$

- ▶ Снова абстракция на верхнем уровне: $\text{intro} \rightarrow$
- ▶ Тип $?_2$ должен иметь стрелочную структуру, запоминаем $?_2 = ?_3 \rightarrow ?_4$
- ▶ Обогащаем контекст

Вывод типа для комбинатора **B** (3)

Правила вывода утверждений типизации в $\lambda \rightarrow$

$$\frac{(x : \sigma) \in \Gamma}{\Gamma \vdash x : \sigma} \text{ ctx} \quad \frac{\Gamma \vdash M : \sigma \rightarrow \tau \quad \Gamma \vdash N : \sigma}{\Gamma \vdash M N : \tau} \text{ elim} \rightarrow \quad \frac{\{x : \sigma\} \cup \Gamma \vdash M : \tau}{\Gamma \vdash \lambda x^{\sigma}. M : \sigma \rightarrow \tau} \text{ intro} \rightarrow$$

$$\frac{\frac{\frac{\frac{\{f : ?_1, g : ?_3, x : ?_5\} \vdash f (g x) : ?_6}{\{f : ?_1, g : ?_3\} \vdash \lambda x. f (g x) : ?_5 \rightarrow ?_6} \text{ intro} \rightarrow}{\{f : ?_1\} \vdash \lambda g x. f (g x) : ?_3 \rightarrow ?_5 \rightarrow ?_6} \text{ intro} \rightarrow}{\emptyset \vdash \lambda f g x. f (g x) : ?_1 \rightarrow ?_3 \rightarrow ?_5 \rightarrow ?_6} \text{ intro} \rightarrow$$

- ▶ Снова абстракция на верхнем уровне: $\text{intro} \rightarrow$
- ▶ Запоминаем: $?_4 = ?_5 \rightarrow ?_6$
- ▶ Обогащаем контекст
- ▶ Обозначим для краткости $\Gamma = \{f : ?_1, g : ?_3, x : ?_5\}$

Вывод типа для комбинатора **B** (4)

Правила вывода утверждений типизации в $\lambda \rightarrow$

$$\frac{(x : \sigma) \in \Gamma}{\Gamma \vdash x : \sigma} \text{ ctx} \quad \frac{\Gamma \vdash M : \sigma \rightarrow \tau \quad \Gamma \vdash N : \sigma}{\Gamma \vdash M N : \tau} \text{ elim} \rightarrow \quad \frac{\{x : \sigma\} \cup \Gamma \vdash M : \tau}{\Gamma \vdash \lambda x^{\sigma}. M : \sigma \rightarrow \tau} \text{ intro} \rightarrow$$

$$\frac{\frac{\frac{\Gamma \vdash f : ?_7 \rightarrow ?_6 \quad \Gamma \vdash g x : ?_7}{\{f : ?_1, g : ?_3, x : ?_5\} \vdash f (g x) : ?_6} \text{ elim} \rightarrow}{\{f : ?_1, g : ?_3\} \vdash \lambda x. f (g x) : ?_5 \rightarrow ?_6} \text{ intro} \rightarrow}{\{f : ?_1\} \vdash \lambda g x. f (g x) : ?_3 \rightarrow ?_5 \rightarrow ?_6} \text{ intro} \rightarrow}{\emptyset \vdash \lambda f g x. f (g x) : ?_1 \rightarrow ?_3 \rightarrow ?_5 \rightarrow ?_6} \text{ intro} \rightarrow$$

- ▶ Аппликация на верхнем уровне: правило $\text{elim} \rightarrow$
- ▶ Тип аргумента неизвестен, заводим мета-переменную $?_7$
- ▶ Типы аргумента и параметра должны совпадать
- ▶ Тип результата функции равен типу аппликации

Вывод типа для комбинатора **B** (5)

Правила вывода утверждений типизации в $\lambda \rightarrow$

$$\frac{(x : \sigma) \in \Gamma}{\Gamma \vdash x : \sigma} \text{ ctx} \quad \frac{\Gamma \vdash M : \sigma \rightarrow \tau \quad \Gamma \vdash N : \sigma}{\Gamma \vdash M N : \tau} \text{ elim} \rightarrow \quad \frac{\{x : \sigma\} \cup \Gamma \vdash M : \tau}{\Gamma \vdash \lambda x^{\sigma}. M : \sigma \rightarrow \tau} \text{ intro} \rightarrow$$

$$\frac{\frac{\frac{\overline{\Gamma \vdash f : ?_7 \rightarrow ?_6}^1 \quad \Gamma \vdash g x : ?_7}{\{f : ?_7 \rightarrow ?_6, g : ?_3, x : ?_5\} \vdash f (g x)} \text{ elim} \rightarrow}{\{f : ?_7 \rightarrow ?_6, g : ?_3\} \vdash \lambda x. f (g x) : ?_5 \rightarrow ?_6} \text{ intro} \rightarrow}{\{f : ?_7 \rightarrow ?_6\} \vdash \lambda g x. f (g x) : ?_3 \rightarrow ?_5 \rightarrow ?_6} \text{ intro} \rightarrow}{\emptyset \vdash \lambda f g x. f (g x) : (?_7 \rightarrow ?_6) \rightarrow ?_3 \rightarrow ?_5 \rightarrow ?_6} \text{ intro} \rightarrow$$

- ▶ В левой ветке правило типизации *ctx*
- ▶ Получаем, что $?_1 = ?_7 \rightarrow ?_6$

Вывод типа для комбинатора **B** (6)

Правила вывода утверждений типизации в $\lambda_{\rightarrow}$

$$\frac{(x:\sigma) \in \Gamma}{\Gamma \vdash x:\sigma} \text{ ctx} \quad \frac{\Gamma \vdash M:\sigma \rightarrow \tau \quad \Gamma \vdash N:\sigma}{\Gamma \vdash M N:\tau} \text{ elim} \rightarrow \frac{\{x:\sigma\} \cup \Gamma \vdash M:\tau}{\Gamma \vdash \lambda x^{\sigma}. M:\sigma \rightarrow \tau} \text{ intro} \rightarrow$$

$$\frac{\overline{\Gamma \vdash f : ?_7 \rightarrow ?_6} \text{ ctx} \quad \frac{\Gamma \vdash g : ?_8 \rightarrow ?_7 \quad \Gamma \vdash x : ?_8}{\Gamma \vdash g \ x : ?_7} \text{ elim } \rightarrow}{\frac{\{f : ?_7 \rightarrow ?_6, g : ?_3, x : ?_5\} \vdash f \ (g \ x)}{\{f : ?_7 \rightarrow ?_6, g : ?_3\} \vdash \lambda x. f \ (g \ x) : ?_5 \rightarrow ?_6} \text{ intro } \rightarrow}{\frac{\{f : ?_7 \rightarrow ?_6\} \vdash \lambda g \ x. f \ (g \ x) : ?_3 \rightarrow ?_5 \rightarrow ?_6}{\emptyset \vdash \lambda f \ g \ x. f \ (g \ x) : (?_7 \rightarrow ?_6) \rightarrow ?_3 \rightarrow ?_5 \rightarrow ?_6} \text{ intro } \rightarrow}$$

- ▶ В правой ветке снова видим аппликацию: $elim \rightarrow$
- ▶ Заводим мета-переменную $?_8$

Вывод типа для комбинатора **B** (7)

Правила вывода утверждений типизации в $\lambda \rightarrow$

$$\frac{(x : \sigma) \in \Gamma}{\Gamma \vdash x : \sigma} \text{ ctx} \quad \frac{\Gamma \vdash M : \sigma \rightarrow \tau \quad \Gamma \vdash N : \sigma}{\Gamma \vdash M N : \tau} \text{ elim} \rightarrow \quad \frac{\{x : \sigma\} \cup \Gamma \vdash M : \tau}{\Gamma \vdash \lambda x^{\sigma}. M : \sigma \rightarrow \tau} \text{ intro} \rightarrow$$

$$\begin{array}{c} \frac{}{\Gamma \vdash f : ?_7 \rightarrow ?_6} \text{ ctx} \quad \frac{\frac{}{\Gamma \vdash g : ?_8 \rightarrow ?_7} \text{ ctx} \quad \Gamma \vdash x : ?_8}{\Gamma \vdash g x : ?_7} \text{ elim} \rightarrow \\ \frac{}{\Gamma \vdash f (g x)} \text{ elim} \rightarrow \\ \frac{\{f : ?_7 \rightarrow ?_6, g : ?_8 \rightarrow ?_7\} \vdash \lambda x. f (g x) : ?_5 \rightarrow ?_6}{\{f : ?_7 \rightarrow ?_6\} \vdash \lambda g x. f (g x) : (?_8 \rightarrow ?_7) \rightarrow ?_5 \rightarrow ?_6} \text{ intro} \rightarrow \\ \frac{\{f : ?_7 \rightarrow ?_6\} \vdash \lambda g x. f (g x) : (?_8 \rightarrow ?_7) \rightarrow ?_5 \rightarrow ?_6}{\emptyset \vdash \lambda f g x. f (g x) : (?_7 \rightarrow ?_6) \rightarrow (?_8 \rightarrow ?_7) \rightarrow ?_5 \rightarrow ?_6} \text{ intro} \rightarrow \end{array}$$

- ▶ В левой ветке первое правило: ctx
- ▶ Получаем $?_3 = ?_8 \rightarrow ?_7$

Вывод типа для комбинатора **B** (8)

Правила вывода утверждений типизации в $\lambda \rightarrow$

$$\frac{(x : \sigma) \in \Gamma}{\Gamma \vdash x : \sigma} \text{ ctx} \quad \frac{\Gamma \vdash M : \sigma \rightarrow \tau \quad \Gamma \vdash N : \sigma}{\Gamma \vdash M N : \tau} \text{ elim} \rightarrow \quad \frac{\{x : \sigma\} \cup \Gamma \vdash M : \tau}{\Gamma \vdash \lambda x^{\sigma}. M : \sigma \rightarrow \tau} \text{ intro} \rightarrow$$

$$\begin{array}{c} \frac{}{\Gamma \vdash f : ?_7 \rightarrow ?_6} \text{ ctx} \quad \frac{\frac{}{\Gamma \vdash g : ?_5 \rightarrow ?_7} \text{ ctx} \quad \frac{}{\Gamma \vdash x : ?_5} \text{ ctx}}{\Gamma \vdash g x : ?_7} \text{ elim} \rightarrow \\ \frac{\Gamma \vdash f : ?_7 \rightarrow ?_6 \quad \Gamma \vdash g x : ?_7}{\{f : ?_7 \rightarrow ?_6, g : ?_5 \rightarrow ?_7, x : ?_5\} \vdash f (g x)} \text{ elim} \rightarrow \\ \frac{\{f : ?_7 \rightarrow ?_6, g : ?_5 \rightarrow ?_7\} \vdash f (g x) : ?_5 \rightarrow ?_6}{\{f : ?_7 \rightarrow ?_6, g : ?_5 \rightarrow ?_7\} \vdash \lambda x. f (g x) : ?_5 \rightarrow ?_6} \text{ intro} \rightarrow \\ \frac{\{f : ?_7 \rightarrow ?_6\} \vdash \lambda g x. f (g x) : (?_5 \rightarrow ?_7) \rightarrow ?_5 \rightarrow ?_6}{\{f : ?_7 \rightarrow ?_6\} \vdash \lambda g x. f (g x) : (?_7 \rightarrow ?_6) \rightarrow (?_5 \rightarrow ?_7) \rightarrow ?_5 \rightarrow ?_6} \text{ intro} \rightarrow \\ \emptyset \vdash \lambda f g x. f (g x) : (?_7 \rightarrow ?_6) \rightarrow (?_5 \rightarrow ?_7) \rightarrow ?_5 \rightarrow ?_6 \end{array}$$

► В правой ветке снова срабатывает первое правило: *ctx*

► Получаем $?_8 = ?_5$

⚡ Заменяем мета-переменные на типовые $(\beta \rightarrow \gamma) \rightarrow (\alpha \rightarrow \beta) \rightarrow \alpha \rightarrow \gamma$

Содержание

1. Идея систем типов
2. $\lambda \rightarrow$ интуитивно
3. $\lambda \rightarrow$ формально
 - ▶ Задача проверки типа
 - ▶ Задача вывода типа
 - ▶ **Задача населения типа**
 - ▶ Стили типизации
4. Материалы (☀)

Простые примеры задач населения

Найдите все возможные варианты для терма M в каждом из случаев:

todo $\{x : \beta\} \vdash M : \alpha \rightarrow \alpha$

todo $\{x : \beta\} \vdash M : \alpha \rightarrow \beta$

todo $\{x : \beta\} \vdash M : \alpha \rightarrow \gamma$

- ▶ Единственный терм типа α передаётся аргументом
- ▶ Терм обязан вернуть свой аргумент: $M = \lambda x. x$

- ▶ Единственное значение типа β есть в контексте
- ▶ Обязаны вернуть значение из контекста $M = \lambda y. x$

- ▶ Значение типа γ брать неоткуда
- ▶ Этот тип не населён в данном контексте

Содержание

1. Идея систем типов
2. $\lambda \rightarrow$ интуитивно
3. $\lambda \rightarrow$ формально
 - ▶ Задача проверки типа
 - ▶ Задача вывода типа
 - ▶ Задача населения типа
 - ▶ Стили типизации
4. Материалы (☀)

Типизация в стиле Карри vs в стиле Чёрча

Правила вывода утверждений типизации в $\lambda \rightarrow$

$$\frac{(x : \sigma) \in \Gamma}{\Gamma \vdash x : \sigma} \text{ ctx} \quad \frac{\Gamma \vdash M : \sigma \rightarrow \tau \quad \Gamma \vdash N : \sigma}{\Gamma \vdash M N : \tau} \text{ elim} \rightarrow \frac{\{x : \sigma\} \cup \Gamma \vdash M : \tau}{\Gamma \vdash \lambda x^{\sigma}. M : \sigma \rightarrow \tau} \text{ intro} \rightarrow$$

По Чёрчу в термах записываются типы при связанных переменных.

- Количество кода

По Чёрчу Программисту приходится больше писать

По Карри Меньше кода, сложнее задача типовой подсистеме языка

- Количество способов приписать терму тип

По Чёрчу Либо 0, либо 1

По Карри Либо 0, либо ∞

- Алгоритм вывода

По Чёрчу Проще, вся необходимая информация написана в терме

По Карри Сложнее, приходится решать систему типовых уравнений

Содержание

1. Идея систем типов
2. $\lambda_{\rightarrow}$ интуитивно
3. $\lambda_{\rightarrow}$ формально
 - ▶ Задача проверки типа
 - ▶ Задача вывода типа
 - ▶ Задача населения типа
 - ▶ Стили типизации
4. Материалы (☀)

Деревья вывода

- ▶ Теория доказательств математической логики — доказываем принадлежность утверждения типизации множеству истинных утверждений
- ▶ Истинное утверждение в теории доказательств — то, которое можно *синтаксически* вывести по правилам вывода из аксиом
- ▶ Деревья вывода — синтаксические, машинно-проверяемые конструкции
- ▶ Деревья сами по себе не несут смысла, наличие корректного дерева говорит только о возможности его построения
- ▶ Чтобы деревья имели смысл, для каждой теории доказываются утверждения вида:
 - ▶ Если существует дерево вывода с корнем X , то верно Y*e.g.* Если существует $\lambda_{\rightarrow}$ дерево с утверждением про терм M в корне (терм M в таком-то контексте имеет такой-то тип), то этот терм строго нормализуем
- ▶ Иногда по правилам вывода очевиден алгоритм построения дерева
 - ▶ В нашем случае (для $\lambda_{\rightarrow}$) это так
 - ▶ В общем случае — нет (математики оказались бы не нужны)

Не всё сильно-нормализуемое типизируется (*)

Вспомним: всё, что типизируется, “хорошее”, но обратное не всегда верно. Будем “хорошим” считать сильную нормализуемость.

2 K терм сильно нормализуем и редуцируется так:

$$\mathbf{2\ K} = \lambda z. \mathbf{K} (\mathbf{K} z) = \lambda z\ a\ b. z$$

Однако протипизировать это не получится.

- ▶ Наиболее общий тип **K** — это $\gamma \rightarrow \tau \rightarrow \gamma$
- ▶ $\mathbf{2} := \lambda s\ z. s\ (s\ z)$, наиболее общий тип **2** — это $(\alpha \rightarrow \alpha) \rightarrow \alpha \rightarrow \alpha$
- ▶ По правилу типизации аппликации, применение **2** к **K** накладывает на γ, τ, α ограничения $\alpha = \gamma, \alpha = \tau \rightarrow \gamma$

Противоречие.

Эквивалентности не сохраняют тип (*)

β -эквивалентность не сохраняет тип

Из лекции известно, что если M редуцируется к N и $\Gamma \vdash M : \tau$, то $\Gamma \vdash N : \tau$.

Неправда, что если M редуцируется к N и $\Gamma \vdash N : \tau$, то $\Gamma \vdash M : \tau$.

Пример: $N = \lambda x. x$, $N : \alpha \rightarrow \alpha$; $M = \lambda x. (\lambda y. x) (x \mathbf{I})$, и за одно редуцирование M переходит в N , но $M : \alpha \rightarrow \alpha$ не выполняется: тогда бы подтерм $x \mathbf{I}$ не типизировался, так как α не является стрелочным типом.

η -эквивалентность не сохраняет тип

Если $\lambda x. M x : \tau$ (где x не является свободным в M), то $M : \tau$.

Если $M : \tau$, то не обязательно $\lambda x. M x : \tau$.

Пример: $\mathbf{I} : \alpha \rightarrow \alpha$, но $\lambda x y. x y : (\beta \rightarrow \gamma) \rightarrow (\beta \rightarrow \gamma)$ нельзя приписать тип $\alpha \rightarrow \alpha$: тогда бы подтерм $x y$ не типизировался, так как α не является стрелочным типом.

Что посмотреть в транспорте

- ▶ Формальная верификация операционных систем

- ▶ Соответствие Карри–Ховарда: от матлогики к программированию
- ▶ Вывод типов от Хиндли-Милнера до GHC 8.8
- ▶ Программирование с зависимыми типами на языке Idris
- ▶ Edwin Brady "Type-Driven Development with Idris"
- ▶ Pierce, Benjamin C. Types and programming languages. MIT press, 2002.
- ▶ Пирс Б. Типы в языках программирования. — Лямбда пресс, Добросвет, 2012. — 656 с.
- ▶ Sandy Maguire "Thinking with Types. Type-Level Programming in Haskell"
- ▶ Henk Barendregt, Wil Dekkers, and Richard Statman. "Lambda Calculus with Types". Cambridge University Press, New York, NY, USA, 2013.
- ▶ Dunfield, Jana, and Neel Krishnaswami. "Bidirectional typing." *ACM Computing Surveys (CSUR)* 54.5 (2021): 1-38.